

CSA17 - Scenario Based Assignment
Assessment Tool 2 (CO2) - Solution

Name: Roshitha

Reg No: 192424400

Q1. AI Drone Delivery in a Flood-Affected Region

Scenario Recap

A drone must deliver emergency medicines through a dynamic environment where roads can become blocked and weather changes movement cost. The drone has a heuristic (straight-line distance), partial real-time blocked-path updates, and variable movement costs.

i. Behaviour of Greedy Best-First Search

Greedy Best-First Search expands the node with the smallest heuristic value $h(n)$ only, ignoring the cost already spent ($g(n)$). In the simulation, from the depot the drone reaches node A, and then — purely because node E has a very low straight-line-distance estimate to the flood zone ($h = 1$) — it commits to the expensive $A \rightarrow E$ edge (cost 9) instead of the cheaper $A \rightarrow C \rightarrow \text{FloodZone}$ route, ending on a total cost of 13 instead of the true optimum of 6.

- Strength: very fast, expands few nodes, well suited to the drone's urgency and limited computation time.
- Weakness under uncertainty: it can be misled by a heuristic that looks attractive but ignores real travel cost, terrain, or weather, so it is not guaranteed to find the safest or cheapest route.

ii. Behaviour of A^*

A^* expands nodes by $f(n) = g(n) + h(n)$, balancing the cost already incurred with the estimated remaining distance. In the same graph, A^* correctly identifies the Depot $\rightarrow A \rightarrow C \rightarrow \text{FloodZone}$ route as optimal (cost 6) even though C's heuristic estimate was inflated, because the accumulated g -cost still kept that branch's f -value competitive until it was fully explored.

iii. Impact of Heuristic Accuracy

Both algorithms are sensitive to heuristic quality, but in different ways. Greedy Best-First Search's final decision depends entirely on $h(n)$, so an inaccurate or overestimated heuristic (as with node C in the simulation) can steer it away from the true best route with no correction mechanism. A^* is more resilient because $g(n)$ continues to anchor the search in real cost, but its optimality guarantee only holds if the heuristic never overestimates the true remaining cost (admissibility); an inflated heuristic can still cause A^* to explore extra nodes or, in the worst case, settle for a suboptimal path if consistency is violated.

iv. Effect of Dynamic Changes (Blocked Paths)

When the previously optimal edge $C \rightarrow \text{FloodZone}$ becomes blocked mid-mission, the simulation shows A^* re-planning from scratch with the updated graph and correctly falling back to the Depot $\rightarrow A \rightarrow E \rightarrow \text{FloodZone}$ route (cost 13) — the best remaining option. This demonstrates that a static, one-shot plan is not sufficient in a dynamic environment; the

search must be re-invoked whenever new information (a blocked road) arrives, which adds computational overhead but is essential for correctness and safety.

v. Recommendation

A* is the more suitable overall algorithm for this scenario: it retains the speed benefit of using a heuristic while still reasoning about real accumulated cost, giving a much better balance of optimality and responsiveness. For extremely time-critical situations where even A*'s search time is too slow, a Greedy search with a well-validated, close-to-admissible heuristic could serve as a fast fallback, but this trades away A*'s optimality and safety guarantees, so A* should be the default and Greedy only a controlled fallback.

Simulation (Program Output)

```
-- Greedy Best-First Search --
Greedy expands Depot (h=8)
Greedy expands A (h=6)
Greedy expands E (h=1)
Greedy expands FloodZone (h=0)
Greedy path: [Depot, A, E, FloodZone] | Total cost: 13

-- A* Search --
A* expands Depot (g=0, h=8, f=8)
A* expands A (g=2, h=6, f=8)
A* expands C (g=4, h=7, f=11)
A* expands FloodZone (g=6, h=0, f=6)
A* path: [Depot, A, C, FloodZone] | Total cost: 6

-- Dynamic change: edge C->FloodZone suddenly blocked --
A* re-planned path: [Depot, A, E, FloodZone] | Total cost: 13
```

Q2. Smart City Traffic Signal Optimization

Formulation as an Optimization Problem

The state is a vector of signal timings across intersections; the objective function combines total waiting time, fuel consumption, and congestion into a single cost to minimize. This is a continuous, high-dimensional local-search / optimization problem rather than a path-finding search: there is no single goal state to reach, only better and worse configurations, and the state space (all possible timing combinations across hundreds of intersections) is far too large for uninformed or informed graph search (BFS, UCS, A*) to explore exhaustively. Local search methods that iteratively improve a candidate solution are the appropriate class of technique here.

i. Hill Climbing in this Scenario

Hill Climbing starts from an initial timing configuration and repeatedly moves to the best neighbouring configuration, stopping as soon as no neighbour improves the objective. The simulation starts at `signal_timing = 20` and, after only two improving moves, gets stuck at 19 with cost 5.707, even though a far better configuration exists near 75 (cost 0.585). Its key limitation is that it only ever accepts improving moves, so it cannot escape a local optimum once every immediate neighbour looks worse.

ii. Local Maxima, Plateaus, and Ridges

- Local maxima/minima: a signal timing that looks optimal locally (e.g., 19 in the simulation) but is far from the true city-wide optimum — in practice this could mean one intersection's timings look fine in isolation while overall congestion stays high.
- Plateaus: regions where small timing changes produce almost no change in waiting time, so Hill Climbing has no gradient to follow and may wander or stop prematurely, mirroring corridors where traffic volume barely differs between two nearby timing plans.
- Ridges: a narrow path of improving configurations that requires several coordinated moves (e.g., adjusting several intersections together) but looks like a series of poor single-step moves, causing simple Hill Climbing to miss it.

iii. Simulated Annealing / Genetic Algorithms as a Fix

Simulated Annealing occasionally accepts a worse move with a probability that depends on a temperature parameter that gradually cools, allowing it to escape local optima early on while still converging to a good solution as the temperature drops. In the simulation, Simulated Annealing starting from the same point (20) escapes the local optimum at 19 and converges to $\text{signal_timing} \approx 75$ with cost 0.585 — more than $9\times$ better than Hill Climbing's result. Genetic Algorithms would instead maintain a population of candidate timing plans and use selection, crossover and mutation to explore multiple regions of the search space in parallel, which is naturally suited to optimizing many intersections at once.

iv. Recommendation

Simulated Annealing (or a Genetic Algorithm for very large, multi-intersection coordination) is the better choice for city-wide signal optimization: both trade a small amount of extra computation for a much stronger ability to escape local optima and adapt as traffic patterns change through the day, whereas plain Hill Climbing is scalable and cheap but too easily trapped to be reliable at city scale.

Simulation (Program Output)

Hill Climbing started at $\text{signal_timing}=20$, got stuck at 19 (cost=5.707) after 2 moves -> local optimum
 Simulated Annealing started at $\text{signal_timing}=20$, converged near 75 (cost=0.585) -> escapes the local optimum

Q3. Autonomous Mars Rover Exploration

i. Why an Online Search Agent is Needed

Offline search assumes the entire environment (map) is known in advance so a complete plan can be computed before any action is taken. The rover does not have this luxury — the Martian terrain around it is unmapped, so it must interleave sensing, planning and acting: sense the immediately visible cells, plan the best next step with what is currently known, move, and repeat. This sense-plan-act loop is exactly what an online search agent provides.

ii. Challenges of Partial Observability and Dynamic Environments

- Partial observability: the rover can only sense adjacent cells, so it may commit to a step whose consequences (an upcoming hazard just out of sensor range) are not yet known, risking wasted moves or unsafe positions.
- Dynamic/unknown environment: even though the physical terrain itself does not move, from the rover's knowledge perspective the 'known map' is constantly changing

as new cells are discovered, so any previously computed plan can become invalid the moment a new hazard is sensed.

- Limited replanning budget: onboard compute and communication delay with Earth mean the rover cannot afford to replan exhaustively after every tiny observation, so replanning must be efficient.

iii. Building and Updating the Internal Model

The rover maintains a `known_map` dictionary that starts containing only the starting cell. After each move it senses its four immediate neighbours and merges any newly observed cell types (free or hazard) into `known_map`. Cells that have not yet been sensed are optimistically treated as free/traversable when planning, and the plan is recomputed (via Uniform Cost Search restricted to `known_map`) after every single sensing update — this is visible in the simulation trace, where the planned route changes each time a new hazard is discovered.

iv. Online Search vs Uninformed and Informed Offline Search

- Uninformed search (e.g., BFS/UCS over a fully known map): guaranteed optimal but requires the whole map upfront, which the rover does not have.
- Informed search (e.g., A* with a heuristic over a fully known map): faster than uninformed search but still assumes complete knowledge before planning starts.
- Online search: trades some efficiency (it may revisit cells or take a locally sub-optimal step before better information arrives) for the ability to operate correctly with no prior map at all, which is the rover's actual situation.

v. Recommended Approach

A repeated Uniform-Cost-Search-based online search agent (sense → plan → act → repeat) is the most suitable approach: it guarantees the cheapest route through whatever is currently known at each step, safely avoids sensed hazards, and adapts immediately as new terrain is discovered — directly matching the rover's requirements for safety, adaptability, and operation under uncertainty.

Simulation (Program Output, abridged)

```
Step 1: at (0,0) -> route through (0,1)...(1,4)...(4,4)
Step 2: at (0,1) -> hazard at (0,3)/(1,1) known, replans via row 0
Step 3: at (0,2) -> hazard at (1,3) sensed, replans down column 2
Step 4: at (1,2)
Step 5: at (2,2)
Step 6: at (2,3)
Step 7: at (2,4)
Step 8: at (3,4)
Final path travelled: [(0,0), (0,1), (0,2), (1,2), (2,2), (2,3), (2,4), (3,4), (4,4)]
```

Q4. University Exam Timetable Generation (CSP)

CSP Formulation

- Variables: one variable per course being examined — in the simulation, AI, DBMS, OS, CN, ML.
- Domains: the set of available exam slots for each course — in the simulation, {Slot1, Slot2, Slot3}; in the full real system this would also include exam-hall and invigilator resources.

- Constraints: (1) no two courses that share a student may be scheduled in the same slot (the CONFLICTS set in the code models this); (2) each exam hall can only host a limited number of exams per slot; (3) prerequisite/ordering constraints for certain subject pairs; (4) faculty-availability constraints restricting which slots a course can use; (5) special-accommodation constraints reserving particular slots or halls for some students.

ii. Backtracking Search in this Scenario

Backtracking assigns one course at a time to a slot, checking as it goes that no already-assigned conflicting course shares that slot; if every option for a course fails, the search backtracks and tries a different slot for a previously assigned course. In the simulation this is combined with Minimum-Remaining-Values (MRV) ordering, so the most constrained course (the one with the fewest legal slots left) is always scheduled next, which tends to expose conflicts early and reduces wasted effort.

iii. Constraint Propagation (Forward Checking)

`forward_check_exam()` removes a just-used slot from the domain of every conflicting course immediately after an assignment, rather than waiting to discover the conflict later by trial and error. This means the search can detect an unsolvable partial assignment (an empty domain appearing for some course) the moment it happens, pruning large parts of the search tree before they are ever explored — substantially improving efficiency as the number of courses grows.

iv. Real-World Challenges and Scalability

- As the number of courses, students and halls grows, the constraint graph becomes dense and highly interconnected, so simple backtracking alone can become slow.
- Additional propagation techniques (arc consistency, constraint graphs restructured by most-constrained-variable heuristics) and value-ordering heuristics (least-constraining-value) become important at scale.
- For very large universities, a hybrid of CSP backtracking with local-search repair (e.g., min-conflicts) is often more scalable than pure backtracking, since it can start from a near-complete timetable and fix violations incrementally rather than rebuilding from scratch.

MRV-guided backtracking with forward checking is recommended as the core strategy, with a local-search repair step layered on top for scaling to very large course/student populations.

Simulation (Program Output)

```
Trying AI -> Slot1
Trying DBMS -> Slot2
Trying OS -> Slot1
Trying CN -> Slot2
Trying ML -> Slot3
```

Final exam timetable:

```
AI: Slot1
DBMS: Slot2
OS: Slot1
CN: Slot2
ML: Slot3
```

Q5. Strategic Game AI (Minimax & Alpha-Beta Pruning)

i. How Minimax Models the Problem

Minimax represents the game as a tree in which the AI (maximizing player) and the opponent (minimizing player) alternate turns; each leaf holds an evaluation-function score for that terminal game state, and the algorithm propagates values upward — the maximizing player always picks the child with the highest value, the minimizing player always picks the child with the lowest value — so the root value represents the outcome both players would reach if both play optimally.

ii. Computational Challenges of Large Game Trees

The number of nodes in a game tree grows exponentially with search depth (branching factor raised to the depth), so for a real-time strategy game with many possible moves per turn, evaluating the full tree to any useful depth within a strict time budget is computationally infeasible with plain Minimax — exactly the concern raised in the scenario.

iii. How Alpha-Beta Pruning Improves Efficiency

Alpha-Beta pruning tracks two bounds while traversing the tree — alpha (the best value the maximizer can already guarantee) and beta (the best value the minimizer can already guarantee) — and cuts off (prunes) any branch that cannot possibly influence the final decision because it is provably worse than an already-found alternative. In the simulation, plain Minimax visits 15 nodes to evaluate the depth-3 tree, while Alpha-Beta pruning reaches the identical result (5) while visiting only 11 nodes, a reduction of 4 nodes on this small tree; on larger, well-ordered trees the saving becomes dramatically larger, which is what makes deeper real-time search feasible.

iv. Designing an Evaluation Function

A practical evaluation function for a real-time strategy game would combine several weighted factors: material/unit advantage (unit count and strength difference between players), positional advantage (map control, resource-tile proximity), economic strength (resources gathered vs. opponent), and threat level (exposure to imminent attack). These are combined into a single numeric score so that Minimax/Alpha-Beta can compare non-terminal states when the full tree cannot be searched to a true endgame, which is necessary because real games are far too deep to solve exhaustively in real time.

v. Balancing Optimality and Real-Time Performance

Recommended strategy: Alpha-Beta pruning combined with iterative deepening (searching progressively deeper until the time budget runs out, always keeping the best result found so far) and good move ordering (trying likely-best moves first to maximize pruning). This gives the AI a strong, time-bounded decision at every turn — sacrificing guaranteed global optimality (which is infeasible at this tree size) in exchange for a good, fast decision, which is the right trade-off for a real-time opponent.

Simulation (Program Output)

```
-- Plain Minimax --  
Minimax result: 5 | nodes visited: 15  
  
-- Minimax with Alpha-Beta Pruning --  
Pruned remaining branches (beta cut-off)  
Pruned remaining branches (alpha cut-off)
```

Alpha-Beta result: 5 | nodes visited: 11
Nodes saved by pruning: 4